

# CIF Microarchitecture Specification

Block: Request Validator

CIF Design Team

June 19, 2026

## Contents

|          |                               |          |
|----------|-------------------------------|----------|
| <b>1</b> | <b>Overview</b>               | <b>2</b> |
| <b>2</b> | <b>Design Parameters</b>      | <b>2</b> |
| <b>3</b> | <b>Internal Blocks</b>        | <b>2</b> |
| 3.1      | Address Range Check . . . . . | 2        |
| 3.2      | Output Mux . . . . .          | 2        |
| <b>4</b> | <b>Interface Signals</b>      | <b>3</b> |
| 4.1      | Inputs . . . . .              | 3        |
| 4.2      | Outputs . . . . .             | 3        |
| <b>5</b> | <b>Functional Behaviour</b>   | <b>3</b> |
| <b>6</b> | <b>Timing Notes</b>           | <b>3</b> |
| <b>7</b> | <b>Edge Cases</b>             | <b>4</b> |

## 1 Overview

The Request Validator is a purely combinational block instantiated once each on the read and write paths. It pops entries from the AR or AW Metadata FIFO, performs a single runtime check (address range validation), and steers the transaction to one of two downstream paths:

- **Valid path:** Transaction forwarded to the Transaction Tag Allocator with `REQ_VALID` asserted.
- **Error path:** Transaction forwarded to the Error Handler with `ERR_VALID` asserted and `AXID` preserved.

The Request Validator contains no storage, no counters, and no state. All protocol-level checks (burst type, WRAP legality, alignment, burst length) are enforced as design-time assertions in the verification environment only. Compliant AXI4 masters are assumed at runtime.

## 2 Design Parameters

| Parameter           | Value | Description                                                          |
|---------------------|-------|----------------------------------------------------------------------|
| <code>ADDR_W</code> | 32    | Address width in bits                                                |
| <code>AXID_W</code> | 6     | AXI ID width                                                         |
| <code>LEN_W</code>  | 8     | Burst length field width ( <code>ARLEN</code> / <code>AWLEN</code> ) |

Table 1: Request Validator parameters

## 3 Internal Blocks

### 3.1 Address Range Check

- **Type:** Purely combinational.
- **Input:** `ADDR[31:0]` from the AR/AW Metadata FIFO.
- **Function:** Checks whether `ADDR` falls within the valid memory address range defined at synthesis time.
- **Output:** `addr_ok` (1-bit) — asserted when the address is valid, deasserted on an address error.
- **Scope:** Address range check only. No burst type, no alignment, no length checks at runtime.

### 3.2 Output Mux

- **Type:** Purely combinational.
- **Function:** Routes the transaction to the correct downstream block based on `addr_ok`.
- **Valid path** (`addr_ok == 1`): asserts `REQ_VALID` to the Tag Allocator; forwards `AXID[5:0]` and `ARLEN/AWLEN[7:0]`.
- **Error path** (`addr_ok == 0`): asserts `ERR_VALID` to the Error Handler; forwards `AXID[5:0]`.
- Only one output path is active per transaction. Both paths are never asserted simultaneously.

## 4 Interface Signals

### 4.1 Inputs

| Signal           | Width | Source              | Description                                   |
|------------------|-------|---------------------|-----------------------------------------------|
| ADDR[31:0]       | 32    | AR/AW Metadata FIFO | Transaction address                           |
| AXID[5:0]        | 6     | AR/AW Metadata FIFO | AXI ID                                        |
| ARLEN/AWLEN[7:0] | 8     | AR/AW Metadata FIFO | Burst length                                  |
| STALL            | 1     | Tag Allocator       | Per-AXID stall; holds output until deasserted |

Table 2: Request Validator input signals

### 4.2 Outputs

| Signal           | Width | Destination   | Description                             |
|------------------|-------|---------------|-----------------------------------------|
| REQ_VALID        | 1     | Tag Allocator | Valid transaction request               |
| AXID[5:0]        | 6     | Tag Allocator | Forwarded AXI ID                        |
| ARLEN/AWLEN[7:0] | 8     | Tag Allocator | Forwarded burst length                  |
| ERR_VALID        | 1     | Error Handler | Address error notification              |
| AXID[5:0]        | 6     | Error Handler | AXI ID preserved for NOP tag allocation |

Table 3: Request Validator output signals

## 5 Functional Behaviour

1. Request Validator pops the head entry from the AR/AW Metadata FIFO: ADDR[31:0], AXID[5:0], ARLEN/AWLEN[7:0].
2. Address Range Check evaluates `addr_ok` combinatorially.
3. If `addr_ok == 1` and STALL is deasserted: Output Mux asserts REQ\_VALID to the Tag Allocator with AXID and ARLEN/AWLEN.
4. If `addr_ok == 0`: Output Mux asserts ERR\_VALID to the Error Handler with AXID. The Error Handler sets `pkt_type = Err_write / Err_read` and injects the transaction into the normal pipeline via the Tag Allocator.
5. If STALL is asserted (Tag Allocator full for this AXID): the Request Validator holds its outputs stable and does not pop the next FIFO entry until STALL deasserts.

## 6 Timing Notes

- Fully combinational. Output (REQ\_VALID or ERR\_VALID) is valid in the same cycle as the FIFO head entry is presented.
- STALL from the Tag Allocator is also combinational — the combined path (FIFO output → Address Range Check → Output Mux → Tag Allocator Stall Logic → STALL back) is a combinational loop that must be timed carefully. A register stage may be inserted between the Request Validator output and the Tag Allocator input if timing closure requires it, at the cost of one cycle of allocation latency.

## 7 Edge Cases

- **Error path stall:** An address error transaction still consumes a Tag Allocator slot. If the AXID is at maximum outstanding (**STALL** asserted), the error path is also held. **ERR\_VALID** is not asserted until **STALL** deasserts.
- **Protocol violations:** Illegal burst type, unaligned WRAP, and maximum burst length violations are not checked at runtime. They are enforced as design-time assertions in the verification environment. A non-compliant master will cause undefined behaviour.
- **Simultaneous valid and error:** Not possible — **addr\_ok** is a single-bit result; exactly one output path is active per transaction.